MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

Group 1

Dương Bình An	Lê Quang Tuyến
Phạm Đức Long	Đỗ Quốc Trung
Dương Hồng Quân	Nguyễn Lê Hồng Nhi

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

Group 1

Dương Bình An	Lê Quang Tuyến
Phạm Đức Long	Đỗ Quốc Trung
Dương Hồng Quân	Nguyễn Lê Hồng Nhi

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

Supervisor:

1. TAN Le Duy

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

Group 1

Dương Bình An — Lê Quang Tuyến
Phạm Đức Long — Đỗ Quốc Trung
Dương Hồng Quân — Nguyễn Lê Hồng Nhi

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an end-to-end fraud risk scoring system built on the IEEE-CIS transaction dataset. Apache Spark performs typed ingestion, quality auditing, chronological splitting, leakage-controlled preprocessing, feature engineering, and Parquet export. A model pipeline compares logistic regression, LightGBM, XGBoost, and CatBoost using ROC-AUC and precision–recall AUC, then packages the selected model with categorical mappings and SHAP explanations. FastAPI exposes scoring, transaction exploration, imports, dataset loading, and human review, while a React dashboard supports investigation. The processed-data contract passed 82 saved verification checks. The V2 LightGBM artifact reports a holdout ROC-AUC of 0.8796 and PR-AUC of 0.4582. The review also identifies limitations: MLflow runs and calibrated thresholds exist for an unpromoted `0.0.2` candidate but are local-only and are not enforced by the backend; heuristic fallback risk, in-memory jobs, manual API type synchronization, and production security gaps also remain. The result is a coherent academic demonstration and a documented path toward production readiness.

Acknowledgments

The members of Group 1 sincerely thank their supervisor, TAN Le Duy, for the guidance and feedback provided during the development and documentation of this project.

The project team consists of Dương Bình An, Lê Quang Tuyến, Phạm Đức Long, Đỗ Quốc Trung, Dương Hồng Quân, and Nguyễn Lê Hồng Nhi. The repository records subsystem responsibilities for data processing and exploratory analysis, model development, backend engineering, and frontend engineering. The interfaces and handover contracts between these areas made it possible to evaluate the system as one end-to-end artifact.

Contents

List of Tables

List of Figures

List of Appendices

Chapter 1

Introduction

1.1 Problem context

Online transaction fraud is a highly imbalanced classification problem in which the positive class is rare, errors have asymmetric consequences, and the data distribution can change over time. A useful operational system must therefore do more than maximize a single offline metric. It must preserve the temporal meaning of validation, make feature transformations reproducible, expose model version and uncertainty, support investigation, and connect predictions to a review process.

The IEEE-CIS Fraud Detection dataset was released through a Kaggle competition and contains anonymized card-not-present transaction and identity information provided by Vesta Corporation (IEEE Computational Intelligence Society, 2019; Amazon Science, 2023). Its scale and mixture of numeric, categorical, missing, and identity attributes make it a useful academic basis for studying big-data preparation and tabular machine learning.

This project addresses the following system question:

> How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring demonstration?

1.2 Objectives

The project has six technical objectives:

1. ingest and validate the full IEEE-CIS transaction and identity files;

2. build leakage-controlled, chronological, model-ready datasets;
3. compare interpretable and gradient-boosted classifiers using metrics suitable for an imbalanced positive class;
4. package the selected estimator behind a stable per-transaction scoring interface with local explanations;
5. expose scoring, persistence, review, import, and dataset loading through a typed API;
6. provide a usable dashboard and reproducible local deployment.

1.3 Research and engineering questions

The implementation and review are organized around four questions:

RQ1: Can the raw data be transformed into a versioned handoff without identifier overlap or training-to-holdout leakage?

RQ2: Which evaluated classifier has the strongest saved validation PR-AUC, and does its final artifact retain performance on the chronological holdout?

RQ3: Can the batch model be served without Spark at request time while retaining categorical compatibility and local explanations?

RQ4: Which gaps prevent the demonstration from supporting a production-readiness claim?

1.4 Contributions

The project contributes:

- a Spark pipeline with typed ingestion, join audits, EDA, temporal splitting, training-only transform fitting, feature contracts, and a machine-readable verifier;
- weighted and deterministic undersampling strategies that leave validation and holdout prevalence untouched;
- a versioned comparison of logistic regression, LightGBM, XGBoost, and CatBoost, with a packaged LightGBM scoring interface;

- a FastAPI and SQL persistence layer for scoring, imports, dataset loads, transaction exploration, and human review;
- a React dashboard with risk visualization, SHAP evidence, data loading, review operations, and fallback-aware states;
- a whole-project audit that separates verified behavior, saved artifact claims, implemented capability, and missing evidence.

1.5 Evaluation basis

The primary predictive metrics are ROC-AUC and precision–recall AUC (PR-AUC). PR curves are particularly informative when positive and negative class counts are highly skewed because precision exposes the false-positive burden among positive predictions (Davis and Goadrich, 2006; Saito and Rehmsmeier, 2015).

The project does not treat area metrics as a complete operational policy. Precision, recall, confusion counts, calibration, error cost, and review capacity at a selected threshold are required before a model score can control real financial decisions.

1.6 Scope and evidence

The evidence snapshot is the repository and locally available generated artifacts inspected on 30 July 2026. Source code proves that a capability is implemented; it does not by itself prove that a training or deployment run completed. Saved JSON/CSV artifacts are reported as *artifact-reported* when they could not be independently regenerated in the review environment.

The thesis excludes:

- claims about real production fraud savings;
- personal or merchant identification from anonymized fields;
- causal explanations of fraud;
- automatic deployment or online learning;
- regulatory approval of the risk-band policy.

1.7 Thesis organization

Chapter 2 introduces the dataset, large-scale processing, boosted-tree models, evaluation, and SHAP. Chapter 3 defines requirements and the integrated architecture. Chapter 4 examines the data pipeline and its leakage controls. Chapter 5 presents model training, results, serving, and limitations. Chapter 6 documents the API, database, frontend, and deployment. Chapter 7 consolidates validation evidence and the production-readiness review. Chapter 8 answers the research questions and identifies future work.

Chapter 2

Background and Related Work

2.1 Transaction fraud as imbalanced learning

Fraud detection differs from a balanced classroom classification task. Fraud is rare, labels can be delayed, behavior changes, and the cost of a missed fraud case may differ substantially from the cost of reviewing a legitimate transaction. Accuracy can therefore be misleading: a classifier that predicts the majority class can appear accurate while detecting no fraud.

For a positive fraud class, precision and recall are:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{2.1}$$

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{2.2}$$

ROC-AUC measures ranking across true- and false-positive rates, while PR-AUC summarizes the precision–recall tradeoff. Davis and Goadrich showed the relationship between ROC and PR spaces and highlighted the usefulness of PR curves under class skew (Davis and Goadrich, 2006). Saito and Rehmsmeier further demonstrated why PR plots can be more informative for imbalanced binary classifiers (Saito and Rehmsmeier, 2015).

2.2 The IEEE-CIS dataset

The source consists of transaction tables and smaller optional identity tables. The public columns include transaction amount, product and card categories, email domains, device attributes, time, and many anonymized groups such as `C`, `D`, `M`, and `V` (IEEE Computational

Intelligence Society, 2019; Amazon Science, 2023). The training transaction file contains 590,540 rows in the inspected source, and the natural fraud prevalence is approximately 3.5%.

The project uses a left join because identity information is unavailable for most transactions. Missingness and identity presence can themselves be useful signals, but they must be interpreted carefully: association does not show that missing data causes fraud.

2.3 Apache Spark for batch processing

Apache Spark provides a unified engine and higher-level APIs for distributed data processing (Zaharia et al., 2016). This project uses Spark for typed CSV ingestion, joins, profiling, feature computation, chronological splitting, Parquet export, and a small decision-tree demonstration.

Spark is not used for request-time inference. Model training crosses from Spark to pandas because the compared Python estimators use in-memory array/data-frame interfaces. This boundary simplifies integration but constrains training scale to available driver memory.

2.4 Gradient-boosted decision trees

Gradient boosting builds an additive ensemble of decision trees, where later trees correct residual errors of the current ensemble (Friedman, 2001). Three modern implementations are compared:

XGBoost provides scalable tree boosting with sparsity-aware learning, weighted quantile sketches, cache-aware access, and distributed execution (Chen and Guestrin, 2016).

LightGBM uses histogram-based learning together with techniques such as gradient-based one-side sampling and exclusive feature bundling (Ke et al., 2017).

CatBoost introduces ordered boosting and categorical-statistic techniques designed to reduce prediction shift and target leakage (Prokhorenkova et al., 2018).

Logistic regression provides a lower-complexity baseline. The comparison uses the same training/validation interface, but the final V1/V2 comparison is not a controlled algorithm-only ex-

periment because feature contracts differ.

2.5 Explainability with SHAP

SHAP is a framework for additive feature attribution based on Shapley values (Lundberg and Lee, 2017). For a prediction $f(x)$, an additive explanation approximates the output as:

$$g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z'_i, \tag{2.3}$$

where ϕ_i is the contribution attributed to feature i. The project uses a tree-specific explainer and returns the five largest absolute local contributions.

SHAP values provide a model explanation, not a causal explanation. They can also explain a different output scale from the probability shown to a user when a separate calibrator transforms the raw model score.

2.6 Service and interface technologies

FastAPI uses Python type annotations and Pydantic models to create runtime validation and OpenAPI schemas (Ramírez). SQLAlchemy maps the transaction and review entities to PostgreSQL or SQLite (Bayer; PostgreSQL Global Development Group). React provides the component model for the browser application (Meta Open Source), while Vite builds and serves the TypeScript bundle (You and Contributors).

The technologies are connected as a modular monorepo rather than independent microservices. This is appropriate for a short project because it reduces deployment complexity, but the backend image inherits heavyweight training dependencies from the model package.

2.7 Related engineering concerns

A production fraud platform would additionally require:

- a feature store or request-time method that exactly reproduces historical aggregates;
- monitored calibration and drift;
- business-approved operating thresholds;
- immutable model lineage and controlled promotion;
- durable case management and review audit history;
- authentication, authorization, secrets, and privacy governance.

The present work evaluates these as readiness gaps instead of claiming that the academic demonstration already provides them.

Chapter 3

Requirements and System Architecture

3.1 Functional requirements

The implemented system satisfies the following demonstration requirements:

Table 3.1: Functional requirements and implementation status

ID	Requirement	Status
FR-01	Validate and process the required transaction and identity CSV files.	Implemented
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.	Verified
FR-03	Preserve a canonical feature order and preprocessing metadata.	Verified
FR-04	Compare multiple binary classifiers using validation metrics.	Artifact-reported
FR-05	Score one transaction and return probability, risk, version, and explanation.	Implemented
FR-06	Persist scored transactions and human review outcomes.	Implemented
FR-07	Filter, sort, paginate, and inspect transactions.	Tested in API suite
FR-08	Import CSV and load processed datasets with progress reporting.	Implemented
FR-09	Support review approval/rejection and a fraud/legitimate label.	Tested in API suite
FR-10	Present risk, SHAP evidence, model state, and fallback state in a browser.	Implemented

3.2 Quality requirements

Table 3.2: Quality requirements and assessment

ID	Requirement	Assessment
QR-01	Temporal leakage control and non-overlapping splits.	Verified
QR-02	Reproducible schema and feature contracts.	Verified
QR-03	Version-aware model artifacts and rollback.	Implemented
QR-04	Explainable tree-model predictions.	Implemented
QR-05	Local deployment with bounded data mounts.	Implemented
QR-06	Automated subsystem tests.	Partial
QR-07	Auditable experiment and promotion lineage.	Incomplete
QR-08	Durable background operations.	Incomplete
QR-09	Authentication, authorization, and audit controls.	Not implemented
QR-10	Production monitoring and service objectives.	Not implemented

3.3 Integrated architecture

Figure 3.1 shows the implemented batch and serving layers. The central contract is the verified model-ready dataset plus its feature and schema metadata.

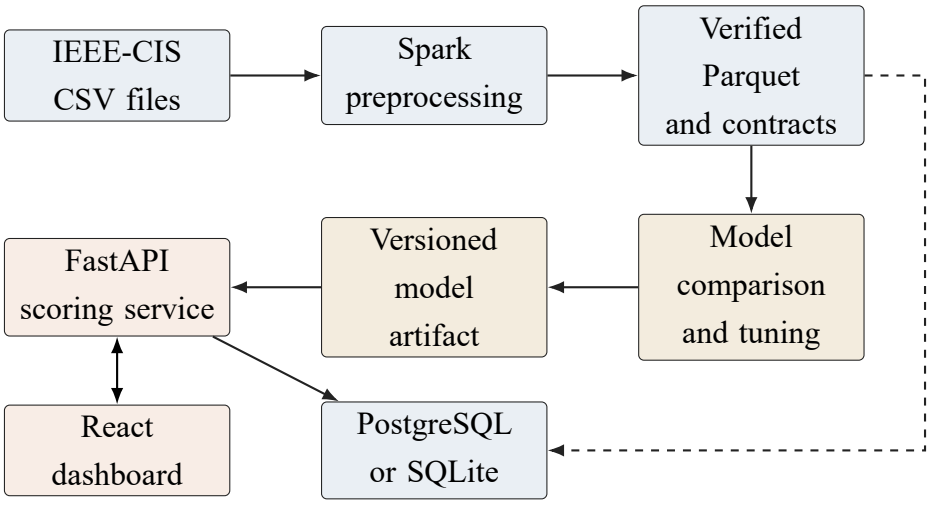

Figure 3.1: End-to-end system architecture. Solid arrows show implemented data or request flows; the dashed path is the processed-dataset loading flow into persistence.

The architecture uses a direct Python dependency between model and backend. This avoids network serialization and a separate model service for the demonstration. It also means backend dependency resolution and image size are coupled to the training project.

3.4 Component responsibilities

3.4.1 Data pipeline

The data component owns raw-file validation, typed ingestion, transaction–identity join preservation, quality and EDA reports, feature engineering, split policy, training-fitted transformations, Parquet output, and the manifest/verifier.

3.4.2 Model package

The model component owns dataset loading, training-only categorical indexing, candidate comparison, tuning, evaluation, artifact packaging, version path selection, SHAP initialization, and the `score(features)` interface.

3.4.3 Backend

The backend owns API validation, risk bands, automatic decisions, model/fallback state, SQL persistence, transaction queries, reviews, CSV import, processed dataset discovery, and background loading jobs.

3.4.4 Frontend

The frontend owns user navigation, filtering and pagination, risk presentation, SHAP visualization and accessible fallback, review input, ad-hoc scoring, data loading/import, polling, theme preferences, and rule-based assistance.

3.5 Primary data flows

3.5.1 Batch preparation

Raw CSV files are mounted into a Linux Spark container. The pipeline writes ignored local artifacts under `data/processed/ieee_cis_fraud_risk`. Downstream modules consume Parquet and JSON contracts, not in-memory Spark objects.

3.5.2 Training

The model loads the weighted training split and untouched validation/holdout splits, fits categorical mappings on training, converts the model matrix to pandas, selects a candidate on validation, and writes a versioned artifact.

3.5.3 Real-time scoring

The backend calls the in-process model package. Raw string categorical values are mapped using dictionaries persisted with the artifact. Probability and local SHAP evidence are mapped into risk presentation and returned through Pydantic schemas.

3.5.4 Review

Scored transactions are stored before investigation. The human review result is a separate entity so an automatic decision remains distinguishable from a reviewer's status and label.

3.6 Contracts

The system has three principal contracts:

1. the model-ready data contract, including canonical feature order and schema/version meta-

data;

2. the model scoring contract, including required/defaultable features, probability, SHAP, and version metadata;
3. the HTTP contract, including transaction, review, dataset, load, job, import, and scoring types.

The first contract is strongly machine-checked. The third is manually duplicated between Pydantic, TypeScript, mocks, and Markdown. At the evidence snapshot, the TypeScript declarations omit several backend metadata fields and `scoring_mode`; OpenAPI-generated types are recommended.

3.7 Risk and trust boundaries

The browser is untrusted input. FastAPI/Pydantic validates JSON and route parameters; CSV parsing applies bounded row limits and per-row error handling. The database and processed-data mount are trusted local resources.

Joblib model artifacts are also a trust boundary because Python deserialization can execute code. Only artifacts produced by a controlled training pipeline should be loaded. Current files have no saved immutable checksum registry.

The fallback heuristic is a further trust concern. It keeps the demonstration available when the model fails, but it must not be allowed to impersonate a production model. The backend exposes a warning, yet readiness remains fail-open.

3.8 Architectural tradeoffs

Table 3.3: Major architectural tradeoffs

Choice	Benefit	Cost
Monorepo and path dependency	Simple handoff and one Compose stack	Backend inherits model dependencies
Spark then pandas	Scalable preparation and familiar estimators	Driver-memory training ceiling
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

Chapter 4

Big-Data Processing and Feature Engineering

4.1 Source validation and typed ingestion

The raw inventory contains the four required IEEE-CIS files and an optional sample-submission file. The four required files occupy approximately 1.29 GB. The pipeline validates discovery, required names, sizes, headers, and schemas before performing expensive Spark actions.

Table 4.1: Observed source files

File	Bytes	Role
`train_transaction.csv`	683,351,067	Labeled transactions
`train_identity.csv`	26,529,680	Optional train identity
`test_transaction.csv`	613,194,934	Unlabeled transactions
`test_identity.csv`	25,797,161	Optional test identity

Explicit Spark schemas avoid inferring a different type across runs and make conversion failures measurable. Identity column names are normalized before the join.

4.2 Join integrity

The transaction table is the authoritative row set and identity is joined with a left join on `TransactionID`. Table 4.2 shows that the join preserves every transaction row.

Table 4.2: Transaction–identity join audit

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	590,540	144,233	446,307	0
Test	506,691	141,907	364,784	0

The large unmatched count is expected. Presence and missingness features preserve information about identity availability without dropping rows.

4.3 Exploratory analysis and quality controls

The pipeline writes source inventory, key and join audits, duplicate summaries, invalid-record summaries, missingness profiles, class distributions, temporal aggregations, product/card/device/email breakdowns, and outlier reports.

These reports are descriptive. A higher observed fraud rate for a device or product group does not establish a causal relationship; the group can be confounded by transaction mix, time, or missingness.

4.4 Leakage-controlled transformation order

Figure 4.1 highlights the transforms whose parameters are fitted only on the training window.

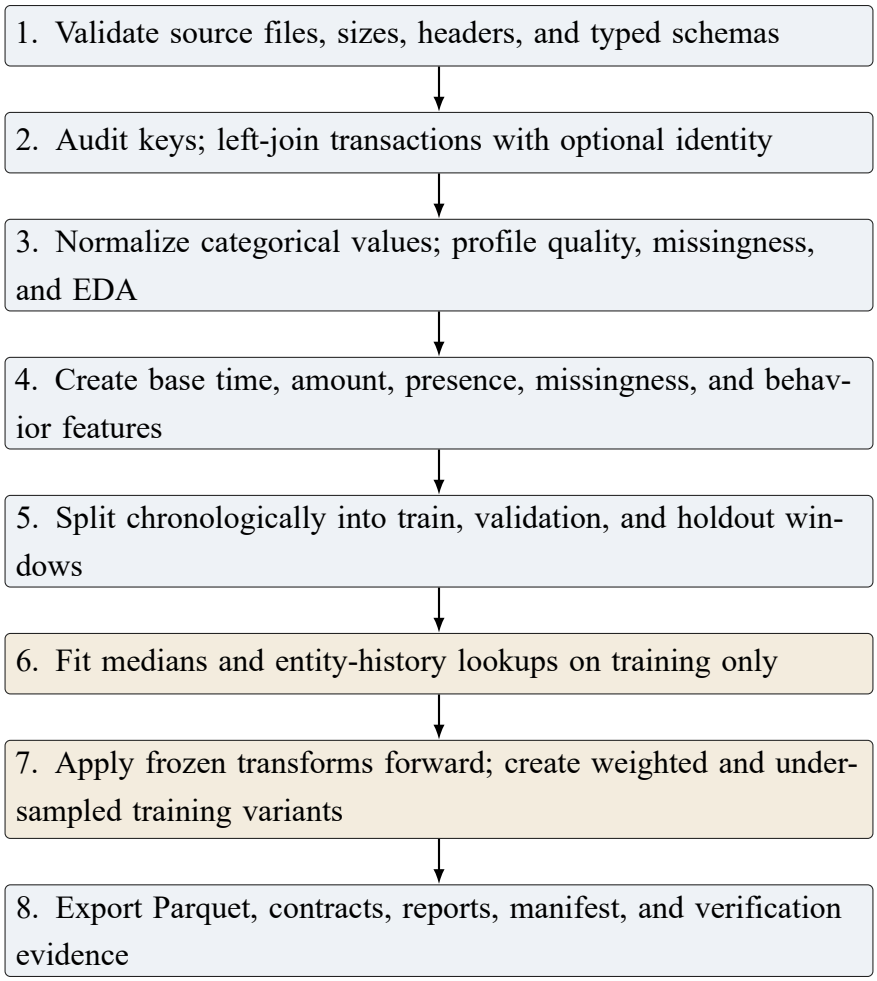

Figure 4.1: Leakage-controlled data pipeline. Gold stages are the training-fitted transformations that must not learn from validation or holdout data.

The use of `TransactionDT` produces forward validation rather than a random split. This better represents the intended question: whether a model trained on earlier transactions ranks later transactions.

4.5 Chronological split

Table 4.3: Chronological labeled splits

Dataset	Rows	Fraud	Fraud rate	`TransactionDT` range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,490	3,036	3.4309%	10,432,915–13,136,653
Holdout	89,094	3,105	3.4851%	13,136,664–15,811,131

The saved verifier found unique identifiers inside each split and zero overlap between split pairs. The nearly stable prevalence across natural windows makes the comparison interpretable, although one validation window cannot quantify variation across multiple historical regimes.

4.6 Feature engineering

4.6.1 Time features

Time features include transaction day, week, hour, a day-of-week proxy, age in days, and a night-transaction flag. The raw `TransactionDT` is retained in the canonical feature order.

4.6.2 Amount features

Amount processing includes the raw value, logarithm, capped value, decimal component, amount band, and ratios/deviations relative to card history. Saved outlier thresholds are learned from training and applied forward.

4.6.3 Missingness and presence

The pipeline produces counts and ratios for selected/identity missing values and flags for identity, device, payer/recipient email, distance, address, and shared email domain presence.

4.6.4 Anonymized behavior fields

Selected `C1--C14`, `D1--D15`, and distance features are retained. Their public semantic meaning is limited, so this report describes them as behavioral/anonymized variables rather than inventing business definitions.

4.6.5 Historical entity features

Card, email, and device keys produce prior transaction counts and amount aggregates. Card history additionally includes standard deviation and time since the previous transaction. Training rows use prior-window history; later splits receive lookups fitted only from the training window.

4.6.6 Categorical features

The seven categorical fields are:

- `ProductCD`, `card4`, and `card6`;
- `DeviceType` and `device_family`;
- `M4` and `amount_band`.

String indexers are fitted on training. The extracted mappings are serialized with the model for Spark-free request-time encoding.

4.7 Missing-value and class-imbalance strategies

Numeric medians and categorical missing policies are persisted. At serving, missing values that remain after categorical mapping use a numeric sentinel. This is compatible with the current tree models, but the sentinel must remain part of the model contract.

The primary training set uses class weights and keeps all 412,956 training rows. A second set undersamples legitimate examples:

Table 4.4: Training and evaluation class distributions

Dataset	Legitimate	Fraud	Legitimate-to-fraud ratio
`train_original`	398,434	14,522	27.44
`train_weighted`	398,434	14,522	27.44
`train_balanced`	43,872	14,522	3.02
`validation`	85,454	3,036	28.15
`holdout`	85,989	3,105	27.69

Validation and holdout remain untouched. This is essential because precision and calibration depend on class prevalence.

4.8 Output contract

The pipeline exports curated data, splits, model-ready Parquet, feature-store tables, preprocessing artifacts, schema artifacts, reports, and demonstration cases. The canonical model-ready datasets are:

- `train_original`;
- `train_weighted`;
- `train_balanced`;
- `validation`;
- `holdout`;
- `kaggle_test`.

4.9 Verification result

The saved verifier reports 82 checks passed and no failures. It validates dataset presence, row counts, identifiers, labels, weights, feature order, schema hashes, chronological order, split overlap, and required artifacts.

One lifecycle inconsistency remains. The regenerated top-level manifest records:

```
verification_pending
```

The dedicated verification report records the later terminal value:

```
ready_for_downstream_training
```

Future finalization should write this terminal verifier result back to the manifest atomically.

4.10 Data limitations

- Feature semantics are partly anonymized.
- Entity aggregates are batch features; a production online path is not implemented.
- The training matrix is collected to pandas before estimator fitting.
- There is one chronological validation window rather than temporal cross-validation.
- Native Windows is not the canonical full Parquet execution path.
- The local processed tree is large and intentionally excluded from Git.

Chapter 5

Model Development, Evaluation, and Serving

5.1 Model-development protocol

The intended training protocol is:

1. read `train_weighted`, validation, and holdout Parquet;
2. fit categorical indexers on training only;
3. apply the frozen indexers to later windows;
4. select the canonical 68-feature order and convert to pandas;
5. compare candidate models on validation;
6. tune the selected tree family on validation;
7. evaluate the final configuration once on holdout;
8. save estimator, mappings, feature order, metadata, and SHAP support.

The exclusion of `split_name`, processing metadata, identifiers, labels, and class weights from the model vector is enforced in `features.py`. This addresses a prior failure in which the string `train_weighted` entered the numeric training frame.

5.2 Candidate models

The baseline is logistic regression. The compared tree-boosting families are LightGBM, XGBoost, and CatBoost. Figure 5.1 presents the saved validation PR-AUC for V1 and V2.

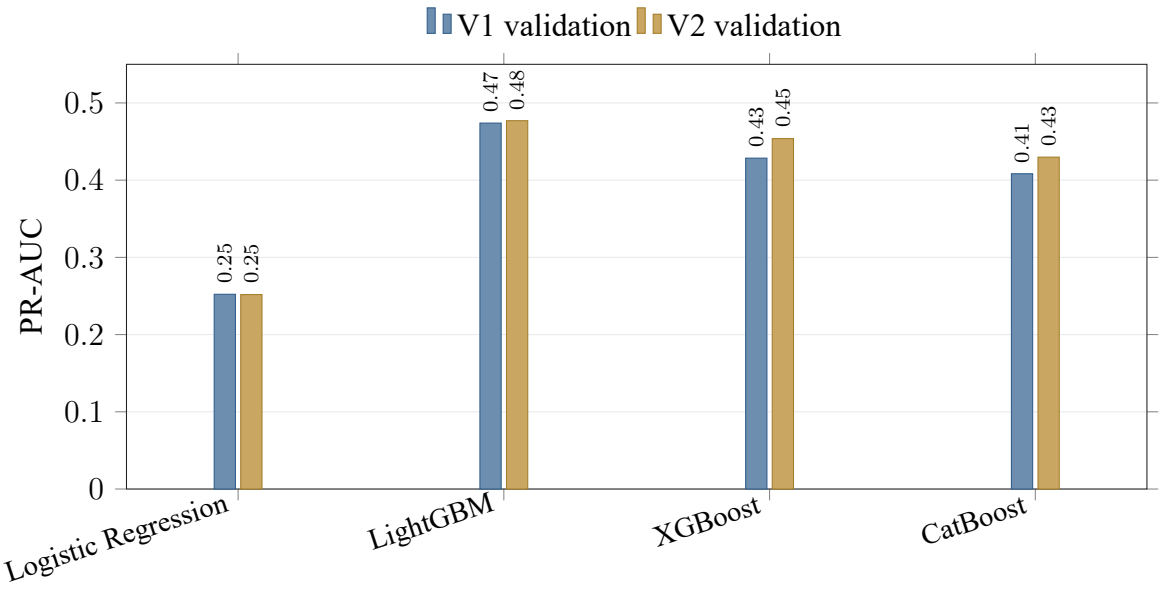

Figure 5.1: Validation PR-AUC by candidate model and artifact version. The chart compares validation results; it does not show a production operating point. Source: saved model-comparison JSON files.

LightGBM ranks first in both saved comparison artifacts. The V2 comparison reports:

Table 5.1: Saved V2 validation candidate metrics

Model	ROC-AUC	PR-AUC
Logistic Regression	0.8092	0.2518
LightGBM	0.8887	0.4770
XGBoost	0.8618	0.4538
CatBoost	0.8445	0.4298

These are ranking metrics on one validation window. They do not define the precision, recall, or review volume produced by the fixed backend bands.

5.3 Tuning

The documented V2 LightGBM grid is intentionally small:

```
n_estimators: [200, 400]
```

```
max_depth: [4, 6]
learning_rate: [0.05, 0.1]
```

The saved best configuration is reported as 400 estimators, depth 6, and learning rate 0.05. A small grid is reproducible and affordable for the project, but it is not evidence that a broad hyperparameter optimum was found.

5.4 V1-to-V2 comparison

Figure 5.2 shows validation and holdout PR-AUC from the final saved artifacts.

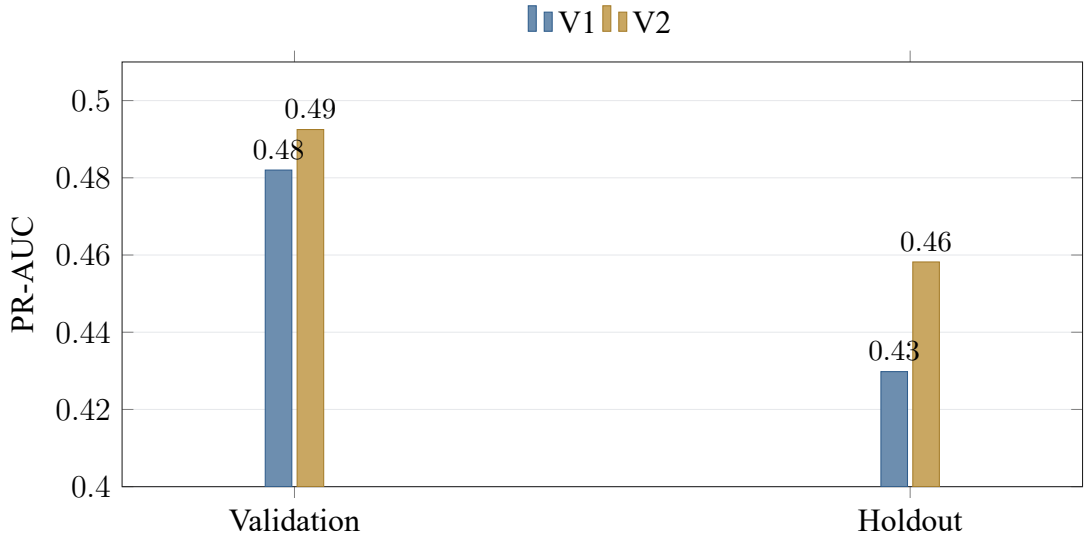

Figure 5.2: V1 and V2 PR-AUC on saved validation and holdout artifacts. The focused vertical scale is declared to make the version differences readable; exact values are printed on each bar.

Table 5.2: Saved final metrics by artifact version

Metric	V1	V2	Absolute change	Relative change
Validation ROC-AUC	0.8877	0.8941	+0.0064	+0.72%
Validation PR-AUC	0.4820	0.4925	+0.0105	+2.18%
Holdout ROC-AUC	0.8684	0.8796	+0.0112	+1.29%
Holdout PR-AUC	0.4298	0.4582	+0.0283	+6.59%

V2 improves all four saved metrics. The largest relative improvement is the holdout PR-AUC. Nevertheless, V1 has 53 features and V2 has 68. The change also includes canonical feature selection and metadata filtering. It is not an ablation study and cannot attribute improvement to one feature group or one hyperparameter.

5.5 Unpromoted `0.0.2` candidate

A later XGBoost candidate on the 68-feature contract records a more complete evaluation workflow than V2: isotonic calibration, selected thresholds, a threshold table, holdout confusion counts, and five finished MLflow stage runs. Table 5.3 summarizes its saved evidence.

Table 5.3: Saved `0.0.2` candidate metrics

Metric	Value
Validation ROC-AUC	0.8982
Validation PR-AUC	0.5050
Holdout ROC-AUC	0.8807
Holdout PR-AUC	0.4318
Precision at review threshold 0.11	0.3275
Recall at review threshold 0.11	0.5578
F1 at review threshold 0.11	0.4127
True negative / false positive	82,433 / 3,556
False negative / true positive	1,373 / 1,732

The candidate selects review and reject thresholds of 0.11 and 0.31 under a minimum-precision rule of 0.30. Its holdout PR-AUC is below V2's 0.4582, so it is evidence that the fuller workflow executed, not evidence that the candidate should replace V2. Calibration and threshold selection also reuse the validation period after model selection used that period; a separate temporal window is needed before treating the operating policy as independently validated.

5.6 Artifact structure and versioning

The current V2 directory contains:

- `baseline_logreg_v2.joblib`;
- `final_model_v2.joblib`;
- `model_comparison_v2.json`;
- `training_metadata_v2.json`.

The `0.0.2` directory additionally contains a calibration model, threshold configuration, threshold-analysis table, validation metrics, and holdout metrics. These files are candidate evidence and are not selected by the default serving configuration.

Serving defaults to V2 in configuration. Setting `FRAUD_MODEL_SERVING_VERSION=v1` selects the V1 path and supports the legacy root artifact.

Training and serving targets are separate. This is a sound deployment boundary: training a new version need not overwrite the served artifact. The remaining lineage gaps are immutable checksums, external registry location, explicit promotion approval, and rollback smoke evidence.

5.7 Request-time scoring

The scoring module loads one artifact into a process cache and initializes a TreeSHAP explainer for supported tree families. For each request it:

1. checks missing required features;
2. maps raw categorical strings using training-derived dictionaries;
3. substitutes the configured missing sentinel;
4. aligns columns to the training order;
5. calls `predict_proba`;
6. optionally applies a calibrator;
7. computes the five largest absolute SHAP contributions;
8. labels the request as full-feature or partial-demo.

This path does not start Spark. It is suitable for a low-latency API process, although inference latency was not saved as evidence.

5.8 Explainability

For tree models, each response can contain:

```
[
  {"feature": "TransactionAmt", "shap_value": 0.41},
  {"feature": "card4", "shap_value": -0.19}
]
```

A positive value pushes the model output toward the fraud class and a negative value pulls it toward legitimate. The list is local to one prediction and must not be interpreted as global feature importance or causal explanation.

When a calibrator transforms raw model probability, the current SHAP values still explain the raw model. A production UI should label this distinction or use an explanation method aligned to the displayed output.

5.9 Risk scoring and operating policy

The backend maps probability to a rounded score and fixed bands. The model code contains threshold-analysis and optional threshold metadata paths. V2 has no calibration/threshold outputs; `0.0.2` has both, but the backend does not use model thresholds as decision authority.

This is the most important distinction between model quality and operational readiness. Area metrics show ranking quality across thresholds. They do not show the reviewers needed at scores 40 and 80. The `0.0.2` confusion counts describe threshold 0.11 only; they do not validate the current backend bands or the reject threshold as a two-stage business policy.

5.10 Experiment tracking

The model package implements an MLflow wrapper and declares MLflow as a current dependency (MLflow Contributors). The observed SQLite database contains five finished `0.0.2` runs:

1. baseline;
2. candidate comparison;
3. tuning/model selection and validation;
4. calibration and threshold policy;
5. final holdout evaluation.

Run parameters and metrics are queryable, but this is still local-only evidence. The artifact URIs point to a container-local `/app/artifacts/mlflow-artifacts` path, and the database has no registered models or model versions. A complete lineage record must additionally persist dataset/schema versions, code revision, artifact checksum, promotion approval, and rollback evidence in a durable shared store.

5.11 Model threats to validity

- Metrics are saved artifacts and were not all independently regenerated during documentation.
- The local review environment initially lacked LightGBM.
- One validation window can favor a transient regime.
- V1/V2 feature sets differ.
- Candidate calibration and threshold selection reuse the validation period after model selection.
- Candidate thresholds are not enforced by the backend, and V2 has no equivalent operating-point artifact.
- Full data is collected to pandas for fitting.
- No production shadow test or delayed-label drift study exists.

5.12 Model-readiness decision

V2 is the current software default and the saved offline champion. It should be described as a rollback-safe candidate rather than a fully governed production model until:

1. a frozen environment loads and scores the artifact without heuristic fallback;

2. candidate operating evidence is reproduced on independent windows and compared with V2;
3. the backend policy uses an approved threshold contract;
4. MLflow lineage, registry/promotion records, and checksums are saved in a durable shared location;
5. shadow behavior and rollback are tested.

Chapter 6

Application and Deployment

6.1 Backend application

The FastAPI application creates SQL tables during startup, loads the scorer, configures CORS, and registers transaction and data/job routers. PostgreSQL is used in Compose; SQLite is the default for local development.

6.1.1 Persistence model

The `Transaction` entity stores:

- transaction identifier and amount;
- raw feature dictionary;
- probability, score, band, and automatic decision;
- SHAP top-five payload;
- model version and score timestamp.

The `Review` entity stores status, fraud/legitimate label, reviewer, note, and update time. A missing review is exposed as `pending`. A subsequent review replaces the existing review for the transaction.

6.1.2 API surface

Table 6.1: Implemented API endpoints

Method	Path	Purpose
GET	`/health`	Process health
GET	`/meta`	Model/schema/feature/band metadata
POST	`/score`	Non-persistent ad-hoc scoring
GET	`/transactions`	Filtered, sorted, paginated list
GET	`/transactions/stats`	Aggregate dashboard statistics
GET	`/transactions/import/template`	Model-aligned CSV template
POST	`/transactions/import`	Bounded synchronous CSV scoring/import
GET	`/transactions/{id}`	Transaction detail
POST	`/transactions/{id}/review`	Human decision and label
GET	`/datasets`	Available processed datasets
POST	`/data/load`	Start a background dataset load
GET	`/jobs/{id}`	Job status
GET	`/jobs/active/current`	Active job or null
POST	`/jobs/{id}/cancel`	Cooperative cancellation

The exact request/response types are reproduced in Appendix A.

6.1.3 Transaction queries

List queries support exact risk/decision/review filters, inclusive score range, exact numeric identifier search, four sort modes, and bounded pagination. SQL aggregates calculate statistics without loading the full table into Python. Every risk band is represented in the statistics response, including zero-count bands.

6.1.4 CSV import

The import route requires a UTF-8 CSV with `TransactionID`. A per-row SQL savepoint means one bad row does not roll back the full batch. Coverage diagnostics report how many model columns are present. This is important because raw Kaggle files do not include every engineered feature.

6.1.5 Dataset loading and jobs

The dataset route discovers model-ready Parquet. A load can use:

head: the first bounded rows;

sample: a deterministic random sample;

coverage: a deliberately non-representative set covering all five risk bands.

Only one in-memory job is active. Status is lost after restart and is not safe across multiple Uvicorn workers.

6.2 Frontend application

6.2.1 Routes and workflows

Table 6.2: Frontend routes

Route	Workflow
`/`	Transaction KPIs, filtering, sorting, and pagination
`/transactions/:id`	Risk, features, SHAP, and review
`/review`	Pending medium/high/critical cases
`/score`	Partial ad-hoc scoring form
`/import`	Dataset loading, job progress, template, and CSV import

TanStack Query manages server state, polling, and invalidation (TanStack). The API abstraction can switch to local JSON mocks through `VITE_USE_MOCKS=true`. Mock scores preserve UI development but are explicitly not model results.

6.2.2 Risk and explanation presentation

Risk badges combine color and text. The score bullet displays the 0–100 scale and band thresholds. The SHAP component uses diverging bars and preserves a numeric table for accessibility.

A null SHAP response renders an explicit unavailable state.

6.2.3 Review experience

The review form requires both an approve/reject action and a fraud/legitimate label. Optional reviewer and note fields are bounded by backend validation. Mutations invalidate detail, list, queue, and statistic queries so the new state appears consistently.

6.2.4 Rule-based assistant

The chat dock parses a limited grammar for tasks such as finding transactions or explaining displayed evidence. It is deterministic code, not a generative AI system. The distinction is important for system scope and risk.

6.2.5 Accessibility and responsive design

The implementation includes visible labels, focus styles, reduced-motion support, semantic text beyond color, system/light/dark themes, and enlarged mobile controls. The project records a manual checklist at widths 375, 768, 1024, and 1440 pixels.

There are no automated component, accessibility, or browser end-to-end tests. TypeScript types are manually synchronized and currently lag several backend metadata fields.

6.3 Deployment

Figure 6.1 shows the default application stack.

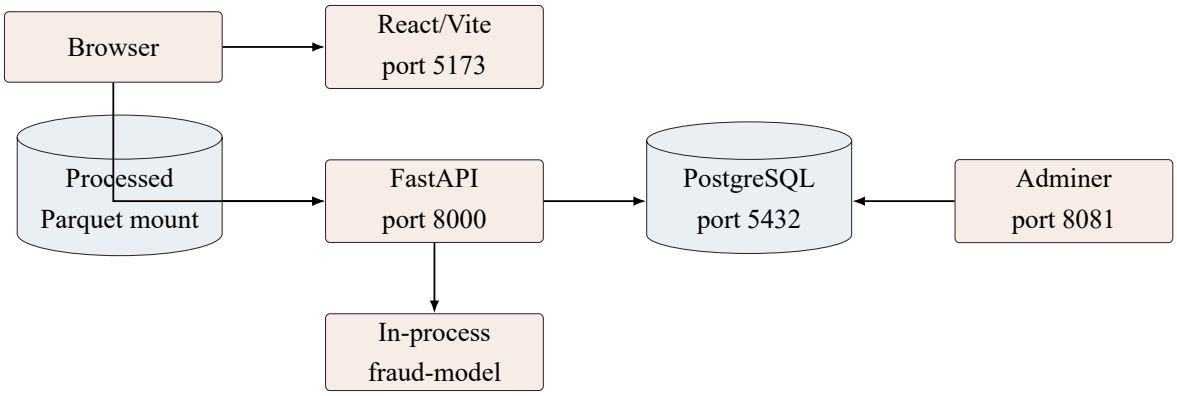

Figure 6.1: Default application deployment. Adminer is optional; Spark training services are a separate Compose profile and are not needed for request-time scoring.

The backend receives processed Parquet through a read-only mount. Model code is installed as a path dependency inside the backend image. Adminer is an optional `tools` profile.

6.3.1 Training profile

The separate `training` profile starts Spark master and worker services plus a model-training container. A local-Spark training service is available when the cluster image cannot be resolved. The Spark UI and Adminer use different host ports (8080 and 8081).

6.3.2 Configuration

Table 6.3: Principal environment variables

Variable	Purpose	Default
`DATABASE_URL`	SQLAlchemy connection	Local SQLite
`CORS_ORIGINS`	Browser origins	Local Vite origins
`DATA_PROCESSED_DIR`	Host data mount	`./data/processed`
`SEED_DATASET`	Dataset for CLI seed	`holdout`
`SEED_LIMIT`	Seed rows	5,000
`MAX_IMPORT_ROWS`	CSV cap	5,000
`FRAUD_MODEL_SERVING_VERSION`	Served model	`v2`

Variable	Purpose	Default
`FRAUD_MODEL_TRAINING_VERSION`	Training target	`v2`
`SPARK_MASTER_URL`	Spark target	Local/profile
`MLFLOW_TRACKING_URI`	Tracking backend	Local unless set

6.4 Application limitations

- no authentication, authorization, rate limiting, or audit event log;
- no Alembic migrations;
- process health is not strict real-model readiness;
- job registry is in memory and single-worker;
- CSV import is synchronous;
- no bulk re-scoring after a model change;
- backend image includes training-heavy dependencies;
- no automated frontend/browser test suite;
- frontend API types are manually maintained;
- default database credentials are development-only.

Chapter 7

Validation and Whole-System Review

7.1 Validation strategy

The review distinguishes five evidence states:

Verified now:	a command completed successfully during the documentation task.
Artifact-reported:	a value was read from a saved machine-readable artifact.
Previously validated:	a dated report records a successful prior run.
Implemented:	source code provides the capability but it was not exercised end to end.
Not evidenced:	neither a successful run nor adequate saved artifact was found.

This prevents the presence of a script or dependency from being reported as a completed experiment.

7.2 Processed-data validation

The data verifier is the strongest machine-readable quality gate. It reports:

- 82 checks passed and zero failed;
- model-ready datasets are non-empty;
- transaction identifiers are unique;
- labeled and unlabeled schemas use the expected target policy;
- class weights appear only in the weighted training variant;
- split order is chronological and split intersections are empty;
- manifest row counts and schema hashes match;

- feature order, medians, and required reports exist.

The stale `verification_pending` value in the top-level manifest is a finalization defect, not a failed data check, because the later verifier report contains the terminal result.

7.3 Static and runtime reports

The dated static report records successful Python compilation, YAML/JSON parsing, Compose configuration validation, leakage-pattern inspection, contract artifact presence, and Docker build-context correction.

The runtime report records the failure/remediation history rather than only the final pass:

1. Windows Spark report paths conflicted with existing directories;
2. Matplotlib attempted to use a GUI backend;
3. Docker build context excluded required source;
4. a long run left an incomplete/stale manifest;
5. normalization and contract regeneration repaired the output;
6. the verifier reached `ready_for_downstream_training`.

This history is valuable evidence because it documents why Docker Linux is the canonical Parquet path and why the contract-finalization utility exists.

7.4 Automated tests

Table 7.1: Automated and manual test coverage

Area	Covered behavior	Gap
Data verifier	Schema, rows, IDs, split order/overlap, artifacts	Full pipeline is expensive
Model features	Training-only indexer, unseen category, alignment	No multi-window validation

Area	Covered behavior	Gap
Model scoring	Probability, SHAP, missing values, fallback	No latency/calibration test
Model quality	Minimum holdout threshold if artifact is loadable	Can skip without artifact
MLflow configuration	Local-directory creation and remote-URI behavior	Full server not exercised
Backend risk	Every band boundary and 0–100 coverage	Fixed-policy only
Backend API	Health, metadata, list/stats/detail, review, score, import	Real model may be replaced
Frontend build/lint	Static typing and source lint	No component or browser tests
Frontend manual QA	Contrast, focus, motion, responsive widths	Not CI-enforced

Only model tests have a GitHub Actions workflow. A whole-system CI gate should also run backend tests, frontend lint/build, API contract generation, processed contract smoke checks, and one real-model scoring smoke.

7.5 Current dependency and execution constraints

At the start of the review, the repository had no local backend/model virtual environment or frontend `node_modules`. The global Python environment contained pandas, Joblib, and PyArrow but not LightGBM. Docker engine access was also denied through the local named pipe.

These are environment constraints rather than direct source defects. They do, however, demonstrate why a frozen container smoke is required before model serving can be claimed independently reproducible.

7.6 Whole-project findings

Table 7.2: Whole-project severity matrix

ID	Finding	Severity	Impact
R1	No authentication, authorization, or audit controls	Critical	Blocks external deployment
R2	Model readiness can fall back to heuristic	High	Predictions can be misidentified
R3	Decision thresholds are not evidence-based	High	Blocks policy approval
R4	Local MLflow runs exist but registry, promotion, and checksum evidence is absent	High	Blocks portable lineage claim
R5	Candidate thresholds are not the served backend policy	High	Blocks operational policy claim
R6	Manifest and verifier status differ	Medium	Handover ambiguity
R7	Jobs and schema lifecycle are non-durable	Medium	Restart/upgrade risk
R8	No automated frontend/browser tests	Medium	UI regression and accessibility risk
R9	API types are manually synchronized and stale	Medium	Client contract risk
R10	Training collects full data to pandas	Medium	Driver-memory ceiling
R11	No bulk re-score/version-isolated views	Medium	Mixed-version analytics

7.7 Threats to validity

7.7.1 Construct validity

ROC-AUC and PR-AUC measure ranking, not business value. The saved results do not establish fraud prevented, investigation effort, or customer impact.

7.7.2 Internal validity

V1 and V2 differ in feature count and preprocessing controls. Their comparison cannot isolate the cause of improvement. A single validation window can also be overfit by repeated model and parameter selection.

7.7.3 External validity

IEEE-CIS is anonymized historical competition data. Behavior, prevalence, labels, feature availability, and latency constraints can differ from a live merchant environment.

7.7.4 Reproducibility

Processed data is locally present and well contracted, but model artifacts are Git-ignored. Five finished `0.0.2` runs are queryable in the local SQLite store, but their artifact URIs are container-local and checksums, registry IDs, promotion approval, and rollback records are absent.

7.8 Production-readiness gates

7.8.1 Gate 1: reproducible serving

1. freeze dependencies;
2. build the backend/model image;
3. load V2 with the expected model version;
4. score a full-feature holdout transaction without fallback;
5. record image digest and model checksum.

7.8.2 Gate 2: approved operating policy

1. reproduce `0.0.2` calibration and threshold evidence on temporal windows not reused for model selection;
2. compare its precision, recall, false positives, false negatives, and calibration with V2;
3. define review capacity and error cost;
4. approve, store, and enforce review/reject thresholds;
5. test exact boundaries through model, backend, and UI.

7.8.3 Gate 3: operational controls

1. fail readiness when the expected model is unavailable;
2. add structured logs, metrics, drift monitoring, and alerting;
3. add migrations and durable job orchestration;
4. add authentication, authorization, audit history, and secret management;
5. add CI across data, model, backend, frontend, and full-stack smoke.

7.8.4 Gate 4: controlled promotion

Run shadow scoring, compare paired predictions, approve the version with a recorded decision, test rollback, and prevent mixed-version historical statistics from being interpreted without a model-version filter or re-score.

7.9 Overall assessment

The system is successful against its academic demonstration objective. All major layers exist and the data handoff is strongly verified. The gap to production is not another dashboard page or another estimator. It is the set of controls that makes an existing prediction trustworthy: reproducibility, operating policy, lineage, readiness, security, and monitoring.

Chapter 8

Conclusion and Future Work

8.1 Answers to the research questions

8.1.1 RQ1: leakage-controlled data handoff

The project produces chronological, non-overlapping model-ready splits and persists the transformation artifacts needed by downstream training. The saved verifier reports 82 passed checks and no failures. The only identified lifecycle defect is the stale terminal status in the top-level manifest.

8.1.2 RQ2: model comparison

LightGBM has the highest saved validation PR-AUC in both V1 and V2 comparisons. The V2 final artifact reports a holdout ROC-AUC of 0.8796 and PR-AUC of 0.4582, improving on V1. Because V1 and V2 differ in feature contract and because V2 lacks operating-point metrics, it should be described as the offline champion rather than proof of production benefit. The later `0.0.2` XGBoost candidate provides calibrated thresholds and confusion counts but has a lower holdout PR-AUC of 0.4318 and has not been promoted.

8.1.3 RQ3: Spark-free serving and explanations

The model package serializes categorical mappings and feature order so FastAPI can score without Spark. Supported tree artifacts return local SHAP contributions. This design successfully separates batch feature preparation from request-time inference, subject to the availability of the same engineered features in a real online environment.

8.1.4 RQ4: production-readiness gaps

The principal gaps are:

- local-only experiment history with no registry/promotion/checksum lineage;
- candidate calibration and thresholds are not the served backend policy;
- fail-open heuristic fallback;
- no authentication, durable jobs, or migrations;
- no drift/latency/service monitoring;
- manual client contract synchronization;
- uneven CI and frontend test coverage.

8.2 Technical contributions

The system demonstrates a complete flow from multi-gigabyte source data to a human review decision:

1. Spark performs typed ingestion, audits, EDA, chronological splitting, and leakage-controlled feature engineering.
2. Explicit contracts make the data–model handoff inspectable.
3. Multiple classifiers are compared under an imbalanced metric.
4. The selected estimator is packaged with categorical mappings and SHAP.
5. FastAPI, SQLAlchemy, and React implement an explainable review workflow.
6. Docker Compose provides local application and training profiles.

8.3 Immediate future work

The first release-hardening milestone should not train a new model. It should make V2 independently reproducible and observable:

1. complete dependency locking and a frozen container smoke;

2. record model checksum, image digest, data/schema versions, MLflow run, and promotion decision in a durable registry;
3. reproduce `0.0.2` operating evidence on independent windows and compare it with V2;
4. make fallback an explicit demo mode and enforce readiness;
5. synchronize/generated API client types and expand CI.

8.4 Model-development future work

After reproducibility:

- perform temporal cross-validation;
- separate model selection, calibration, threshold selection, and holdout;
- run feature-group ablations;
- evaluate paired V1/V2 errors by segment;
- measure inference latency and resource consumption;
- monitor delayed-label performance and drift.

8.5 Platform future work

A deployable platform requires:

- authentication, authorization, secrets, and immutable audit events;
- Alembic migrations and durable background queues;
- a serving-only model dependency profile;
- production metrics, logs, tracing, readiness, and alerts;
- bulk re-scoring or version-isolated reporting;
- retention, privacy, and incident-response policies.

8.6 Final statement

The project is a coherent and well-integrated academic fraud risk scoring demonstration. Its strongest contribution is not a single metric; it is the connection of data contracts, a trained model, explanations, API behavior, persistence, and human review. The whole-project audit defines the additional evidence and controls required to turn that demonstration into a trustworthy deployment candidate.

Bibliography

Amazon Science. Fraud dataset benchmark: Ieee-cis fraud detection, 2023. URL https://github.com/amazon-science/fraud-dataset-benchmark.

Michael Bayer. Sqlalchemy documentation. URL https://docs.sqlalchemy.org/.

Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

Jesse Davis and Mark Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240, 2006. doi: 10.1145/1143844.1143874.

Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

IEEE Computational Intelligence Society. Ieee-cis fraud detection, 2019. URL https://www.kaggle.com/competitions/ieee-fraud-detection/overview.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

Meta Open Source. React documentation. URL https://react.dev/.

MLflow Contributors. Mlflow documentation. URL https://mlflow.org/docs/latest/.

PostgreSQL Global Development Group. Postgresql documentation. URL https://www.postgresql.org/docs/.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html.

Sebastián Ramírez. Fastapi documentation. URL https://fastapi.tiangolo.com/.

Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3): e0118432, 2015. doi: 10.1371/journal.pone.0118432.

TanStack. Tanstack query documentation. URL https://tanstack.com/query/latest.

Evan You and Vite Contributors. Vite documentation. URL https://vite.dev/.

Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016. doi: 10.1145/2934664.

Chapter A

API Reference

A.1 Common enums

```
RiskBand = low | guarded | medium | high | critical
Decision = approve | review | reject
ReviewState = pending | approved | rejected
ReviewLabel = fraud | legit
ScoringMode = full_feature | partial_demo
```

A.2 Core response fields

Table A.1: Core API fields

Field	Type	Meaning
`fraud_probability`	float	Model/fallback output in $[0, 1]$
`risk_score`	integer	Rounded probability times 100
`risk_band`	enum	Fixed backend score band
`decision`	enum	Automatic backend policy result
`scoring_mode`	enum	Full vector or partial demonstration
`shap_top5`	list/null	Local model contributions when available
`model_version`	string	Artifact version used for the score
`processing_version`	string/null	Data processing lineage
`feature_schema_version`	string/null	Feature contract lineage

A.3 Endpoint summary

Method	Path	Response
GET	`/health`	Process status
GET	`/meta`	Model and feature metadata
POST	`/score`	Score response
GET	`/transactions`	Paginated transactions
GET	`/transactions/stats`	Aggregate statistics
GET	`/transactions/import/template`	UTF-8 CSV
POST	`/transactions/import`	Import outcome and coverage
GET	`/transactions/{id}`	Transaction detail
POST	`/transactions/{id}/review`	Updated detail
GET	`/datasets`	Available datasets
POST	`/data/load`	Background job
GET	`/jobs/{id}`	Job state
GET	`/jobs/active/current`	Job or null
POST	`/jobs/{id}/cancel`	Updated job

The normative, implementation-aligned contract is maintained in `docs/API_CONTRACT.md`.

Chapter B

Feature and Risk Contracts

B.1 Canonical feature groups

Table B.1: Canonical feature groups

Group	Representative features
Amount	`TransactionAmt`, log/capped/decimal amount, amount band, card-mean ratios and deviations
Time	`TransactionDT`, day, week, hour, day-of-week proxy, age, night flag
Missingness	Selected/identity missing counts and ratios
Presence	Identity, device, payer/recipient email, distance, address, same-domain flags
Anonymous behavior	`dist1`, `dist2`, selected `C1--C14`, selected `D1--D15`
Card history	Prior count/sum/mean/stddev and time since previous transaction
Email history	Prior count/sum/mean
Device history	Prior count/sum/mean
Categorical	`ProductCD`, `card4`, `card6`, `DeviceType`, `device_family`, `M4`, `amount_band`

B.2 Forbidden model columns

```
TransactionID
isFraud
class_weight
split_name
```

```
processing_version
feature_schema_version
generated_at
```

B.3 Risk bands

Minimum	Maximum	Band	Decision
0	19	low	approve
20	39	guarded	approve
40	59	medium	review
60	79	high	review
80	100	critical	reject

The risk bands are a fixed demonstration policy. They are not derived from the saved V2 validation operating point.

Chapter C

Reproducible Runbook

C.1 Raw data placement

```
data/data/ieee-fraud-detection/
  train_transaction.csv
  train_identity.csv
  test_transaction.csv
  test_identity.csv
  sample_submission.csv # optional
```

C.2 Preprocessing

```
docker compose -f docker-compose.preprocessing.yml build
docker compose -f docker-compose.preprocessing.yml run --rm
   preprocess
docker compose -f docker-compose.preprocessing.yml run --rm verify-
   processed
```

If a completed Spark export lacks a finalized manifest:

```
python -m pipeline.processed_contract \
  --output-dir data/processed/ieee_cis_fraud_risk
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk
```

C.3 Model training

```
cd model
uv sync --frozen
uv run python -m fraud_model.train_baseline
uv run python -m fraud_model.train_compare
uv run python -m fraud_model.tune_and_explain
```

The commands must run in order for one version. Holdout evaluation should occur only after model/parameter selection is fixed.

C.4 Application

```
docker compose up --build
```

Then open:

- dashboard: `http://localhost:5173`;
- API documentation: `http://localhost:8000/docs`;
- optional Adminer: `http://localhost:8081`; select the `PostgreSQL` system, then use the configured database credentials.

C.5 Local development

```
# Terminal 1
cd backend
uv sync --frozen
uv run uvicorn fraud_backend.main:app --reload

# Terminal 2
cd frontend
npm ci
npm run dev
```

C.6 Rollback

```
FRAUD_MODEL_SERVING_VERSION=v1
```

Restart the backend and verify `/meta` before accepting traffic.

Chapter D

Test and Contribution Matrix

D.1 Recommended validation commands

```
# Processed data (read-only unless --write-report is supplied)
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk

# Model
cd model
uv run ruff check .
uv run pytest -q

# Backend
cd backend
uv run ruff check src tests
uv run pytest -q

# Frontend
cd frontend
npm run lint
npm run build
python scripts/check-ui.py
```

D.2 Acceptance scenarios

Table D.1: Acceptance scenarios

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	`/meta` returns expected version with no warning
Full-feature score	`scoring_mode=full_feature`; probability bounded
Partial score	`partial_demo` is visible; required fields enforced
Unseen category	Request succeeds with training-defined unknown mapping
SHAP absent	UI renders explicit unavailable state
Review update	Status/label persist and queue/statistics invalidate
CSV row error	Good rows commit; bad row appears in bounded error list
Job reconnect	Active job is returned after browser refresh
Model rollback	V1 loads and `/meta` reports V1

D.3 Team roster and repository role mapping

Table D.2: Group 1 roster and recorded subsystem contributions

Team member	Repository evidence
Dương Bình An	Data processing, EDA, feature engineering, contracts, and handover
Lê Quang Tuyến	Named team member; a subsystem-specific role is not stated in the reviewed repository
Phạm Đức Long	React dashboard, risk/SHAP presentation, review UX, and import UI
Đỗ Quốc Trung	FastAPI, SQL persistence, imports, background data loading, and Docker
Dương Hồng Quân	Model training, comparison, SHAP, artifact packaging, and scoring
Nguyễn Lê Hồng Nhi	Named team member; a subsystem-specific role is not stated in the reviewed repository

The full roster and supervisor were supplied by the project team. Detailed contribution statements should be confirmed by all members before submission.